Case study: Amazon Fine Food Reviews

```
reviews = pd.read_csv("./Reviews.csv")
```
✓ 3.3s Python

```
reviews.head()
```
✓ 0.0s Python

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	5	1303862400	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0	1	1346976000	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	1	4	1219017600	"Delight" says it all	This is a confection that has been around a fe...
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	3	2	1307923200	Cough Medicine	If you are looking for the secret ingredient i...
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	0	5	1350777600	Great taffy	Great taffy at a great price. There was a wid...

```
reviews.describe()
```
✓ 0.0s

	Id	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time
count	568454.000000	568454.000000	568454.00000	568454.000000	5.684540e+05
mean	284227.500000	1.743817	2.22881	4.183199	1.296257e+09
std	164098.679298	7.636513	8.28974	1.310436	4.804331e+07
min	1.000000	0.000000	0.00000	1.000000	9.393408e+08
25%	142114.250000	0.000000	0.00000	4.000000	1.271290e+09
50%	284227.500000	0.000000	1.00000	5.000000	1.311120e+09
75%	426340.750000	2.000000	2.00000	5.000000	1.332720e+09
max	568454.000000	866.000000	923.00000	5.000000	1.351210e+09

```
reviews.shape
```
✓ 0.0s

```
(568454, 10)
```

```
reviews.columns
```
✓ 0.0s

```
Index(['Id', 'ProductId', 'UserId', 'ProfileName', 'HelpfulnessNumerator',
       'HelpfulnessDenominator', 'Score', 'Time', 'Summary', 'Text'],
      dtype='object')
```

About Dataset

Context

This dataset consists of reviews of fine foods from amazon. The data span a period of more than 10 years, including all ~500,000 reviews up to October 2012. Reviews include product and user information, ratings, and a plain text review. It also includes reviews from all other Amazon categories.

Contents

- Reviews.csv: Pulled from the corresponding SQLite table named Reviews in database.sqlite
- database.sqlite: Contains the table 'Reviews'

Data includes:

- Reviews from Oct 1999 - Oct 2012
- 568,454 reviews
- 256,059 users
- 74,258 products
- 260 users with > 50 reviews

Amazon-Fine-Food-Reviews

Amazon Fine Food Reviews Analysis Data Source: https://www.kaggle.com/snap/amazon-fine-food-reviews

The Amazon Fine Food Reviews dataset consists of reviews of fine foods from Amazon.

Number of reviews: 568,454 Number of users: 256,059 Number of products: 74,258 Timespan: Oct 1999 - Oct 2012 Number of Attributes/Columns in data: 10

Attribute Information:

1. Id
2. ProductId - unique identifier for the product
3. UserId - unqiue identifier for the user
4. ProfileName
5. HelpfulnessNumerator - number of users who found the review helpful
6. HelpfulnessDenominator - number of users who indicated whether they found the review helpful or not
7. Score - rating between 1 and 5
8. Time - timestamp for the review
9. Summary - brief summary of the review
10. Text - text of the review

Objective: Given a review, determine whether the review is positive (Rating of 4 or 5) or negative (rating of 1 or 2).

```
import sqlite3
import nltk
import string
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import confusion_matrix
from sklearn import metrics
from sklearn.metrics import roc_curve, auc
from nltk.stem.porter import PorterStemmer
```

✓ 0.0s

```
con = sqlite3.connect("./finefood_reviews.sqlite")
```

✓ 0.0s

```
filtered_data = pd.read_sql_query("""
SELECT * FROM Reviews
WHERE Score != 3
""", con=con)

# we'll ignore those with score 3 (neither positive, nor negative for sure)
```

✓ 2.4s

```
filtered_data.shape
```

✓ 0.0s

```
(525814, 10)
```

```
# we'll replace the score column with positive/ negative instead of 1-5
def rate_function(score):
    if (score > 3):
        return 'positive'
    else:
        return 'negative'

numeric_score = filtered_data['Score']
positiveNegativeScore = numeric_score.map(rate_function)
filtered_data['Score'] = positiveNegativeScore
filtered_data.head()
```

✓ 0.0s Python

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	positive	1303862400	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0	negative	1346976000	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	1	positive	1219017600	"Delight" says it all	This is a confection that has been around a fe...
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	3	negative	1307923200	Cough Medicine	If you are looking for the secret ingredient i...
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	0	positive	1350777600	Great taffy	Great taffy at a great price. There was a wid...

```
# identifying duplicate data
pd.read_sql_query("""
    SELECT UserId, ProductId, COUNT(*)
    FROM Reviews
    GROUP BY UserId, ProductId
    HAVING COUNT(*) > 1;
""", con)
```

✓ 0.8s

	UserId	ProductId	COUNT(*)
0	A102TGNH1D915Z	B00008DFK5	2
1	A102TGNH1D915Z	B0002DHNXC	2
2	A102TGNH1D915Z	B0009YD7P2	2
3	A102TGNH1D915Z	B000SP1CWW	2
4	A106N4B4EG4DR8	B0008IT4OM	2
...	...	...	...
5854	AZYMD9P9F9UZ6	B000GW46DE	3
5855	AZYMD9P9F9UZ6	B000GW6786	3
5856	AZYMD9P9F9UZ6	B000PMC7GI	3
5857	AZYMD9P9F9UZ6	B000PMHLZK	3
5858	AZYMD9P9F9UZ6	B000PMJLJO	3

5859 rows × 3 columns

```
# identifying duplicate data
pd.read_sql_query("""
    SELECT UserId, Time, COUNT(DISTINCT ProductId) AS NumProducts
    FROM Reviews
    GROUP BY UserId, Time
    HAVING COUNT(DISTINCT ProductId) > 1;
""", con)
# same user, at the same time, but diff product ids
# that's cause the varieties of same product are grouped together
```

✓ 0.7s

	UserId	Time	NumProducts
0	#oc-R115TNMSPFT9I7	1331510400	2
1	#oc-R11D9D7SHXIJB9	1342396800	3
2	#oc-R11DNU2NBKQ23Z	1348531200	2
3	#oc-R11O5J5ZVQE25C	1346889600	3
4	#oc-R12KPBODL2B5ZD	1348617600	2
...	...	...	...
76785	AZZNK89PXD006	1269648000	3
76786	AZZTH6DJ0KSIP	1304208000	2
76787	AZZU1VEO8KUXH	1317513600	3
76788	AZZU4D6TZ2L6J	1247875200	2
76789	AZZV9PDNMCOZW	1329436800	2

76790 rows × 3 columns

```
sorted_data = filtered_data.sort_values(by='ProductId', axis=0, ascending=True)
```
✓ 0.4s

```
# drop duplicates
final_data = sorted_data.drop_duplicates(subset={'UserId', 'Time', 'Text'}, keep='first', inplace=False)
final_data.shape
```
✓ 0.6s

```
(364133, 10)
```

```
364133/525814
```
✓ 0.0s

```
0.6925129418387491
```

roughly 69% of the data remains after dropping duplicates

```
# also it was observed that there are rows where helpfulness numerator (positive) > denominator (total)
# which isn't possible
pd.read_sql_query("\
                  SELECT * FROM Reviews\
                  WHERE HelpfulnessNumerator > HelpfulnessDenominator\
                  ", con=con)
```
✓ 0.2s Python

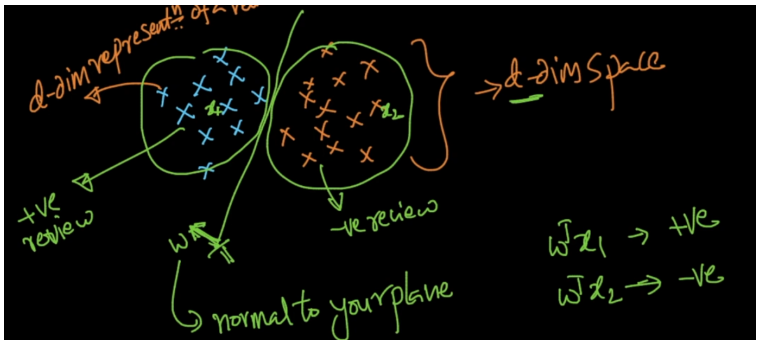
	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
0	44737	B001EQ55RW	A2V0I904FH7ABY	Ram	3	2	4	1212883200	Pure cocoa taste with crunchy almonds inside	It was almost a 'love at first bite' - the per...
1	64422	B000MIDROQ	A161DK06JJMCYF	J. E. Stephens "Jeanne"	3	1	5	1224892800	Bought This for My Son at College	My son loves spaghetti so I didn't hesitate or...

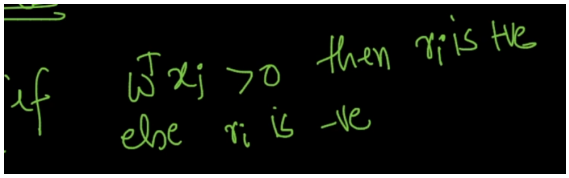
```
# we'll remove those
final_data = final_data[final_data.HelpfulnessNumerator <= final_data.HelpfulnessDenominator]
```
✓ 0.0s

```
print(final_data.shape)
print(final_data["Score"].value_counts())
```
✓ 0.0s

```
(364131, 10)
positive    307026
negative     57105
Name: Score, dtype: int64
```

we have the 'Text' as the most important feature in this data, as we'll be doing a sentiment analysis.
we could convert the text/ words to vectors
say there's some d dim data
if we could draw some hyperplane to divide positive and negative reviews, we could easily decide.





we need to convert reviews/ words to vectors. there ought to be some order to it.

say we have reviews r1, r2, r3
say r1 and r2 are similar semantically, r1 and r3 are not
then the corresp. vectors v1 & v2 ought to be closer (less distance betn them)
similar(r1, r2) > similar(r1, r3)
d(v1, v2) < d(v1, v3)

for this, there are multiple techniques like
Bag of Words (BoW)
tf-idf
Word2Vec
Avg W2V
tfidf w2v

Bag of Words (BoW)

The bag-of-words model is commonly used in methods of document classification where, for example, the (frequency of) occurrence of each word is used as a feature for training a classifier.[2]

Tokenization:
    Break down the text into individual words or tokens.
    This is the process of converting a text into a list of words.

Lowercasing:
    Convert all words to lowercase to ensure that the model treats words like "apple" and "Apple" as the same.

Stopword Removal:
    Remove common words that don't carry much meaning, such as "the," "is," "and," etc. These are called stopwords.

Stemming:
    "beautiful" -> "beauty"

Lemmatization

Building the Vocabulary:
    Create a unique set of words from the entire text dataset. This set of words forms the vocabulary.

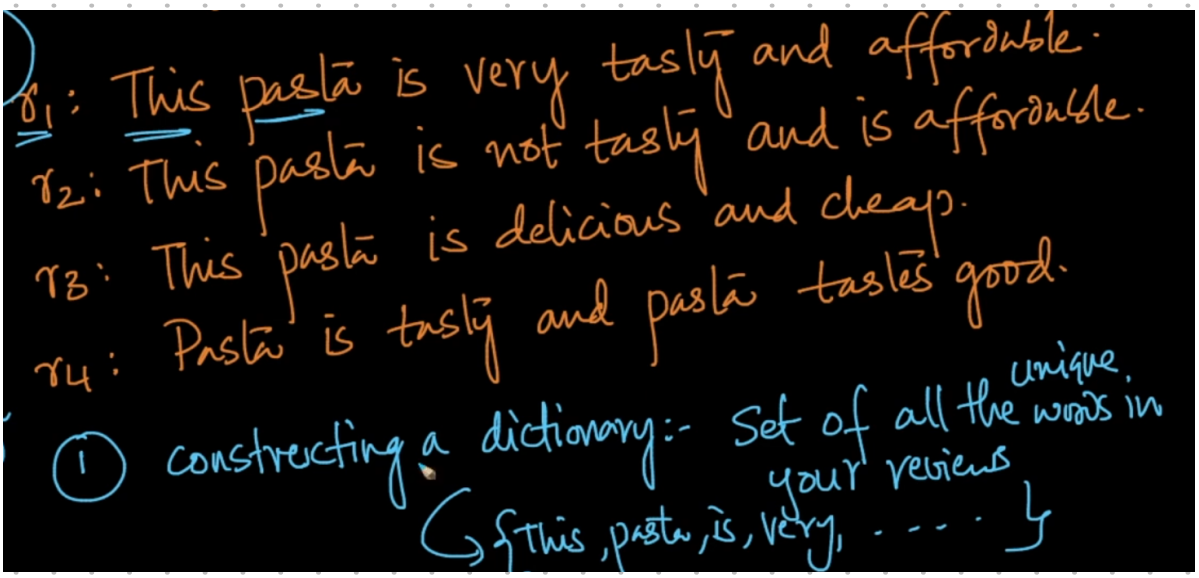
Vectorization:
    Represent each document (a piece of text) as a vector in a high-dimensional space.
    The length of the vector is equal to the size of the vocabulary,
    and each element represents the count (frequency) of a word in the document.

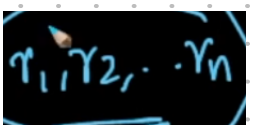
Sparse Representation:
    Most documents will only contain a small subset of the entire vocabulary,
    resulting in a sparse matrix where the majority of the entries are zero.

```
# the sentence "Squealing suitcase squids are not like regular squids."
could be changed into the following BoW dictionary:

{'squeal': 1, 'like': 1, 'not': 1, 'suitcase': 1, 'be': 1, 'regular': 1, 'squid': 2}
```

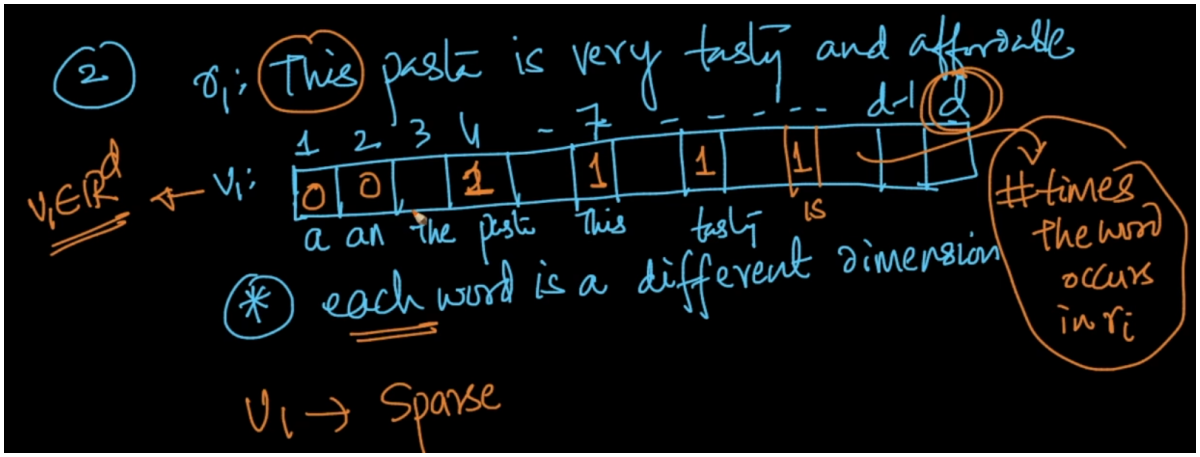
set of all these documents is called a corpus



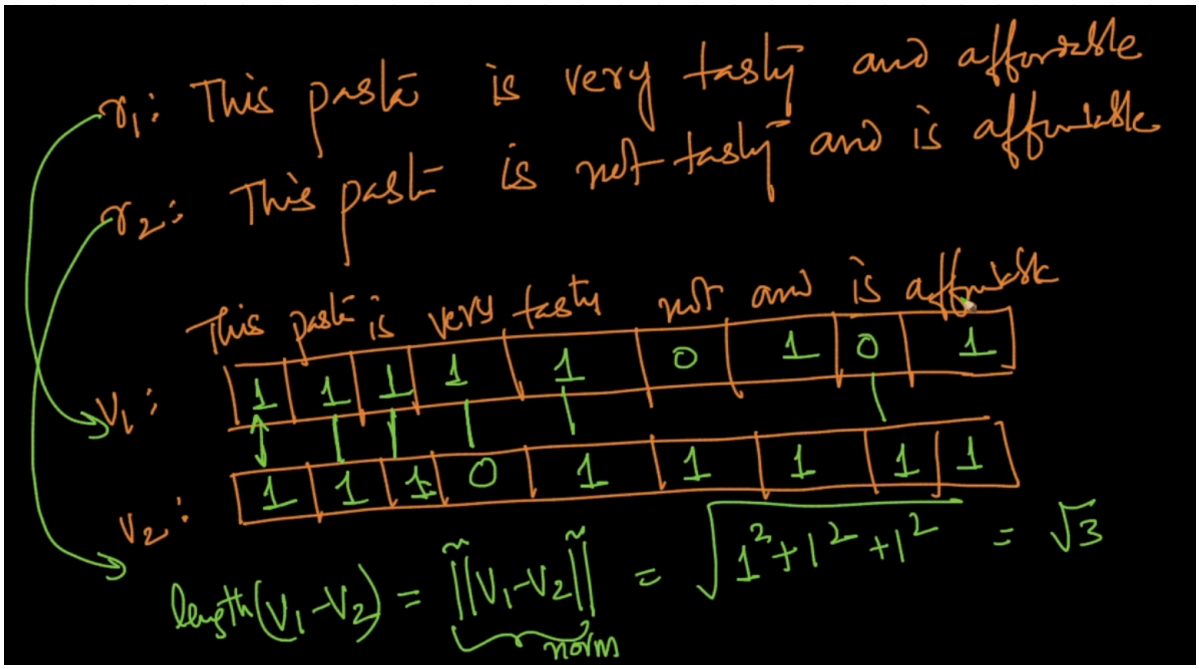


say we have n documents
and d unique words

for a document, we'll create a vector of size d
each index will corresp to a word



there'll be a lot of words in total, and one document will only contain a few
so it'll be a sparse vector



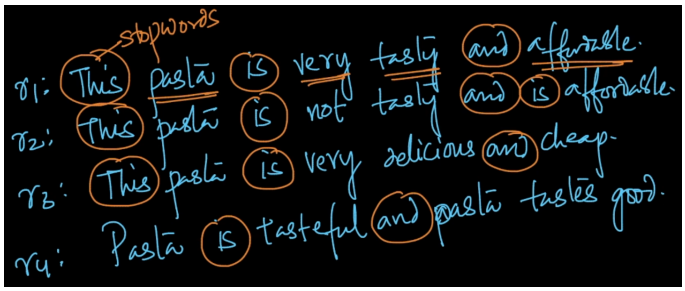
semantic meaning is totally diff (one says tasty, one says not tasty)
but still distance is quite closer
(the pasta is not tasty and the pasta is very tasty both are quite same)
so that's bad

Binary/Boolean Bag of Words

Instead of using frequency (count) we'll just use 1/0 (if the word is present/absent)
so ||v1-v2|| ~= sqrt(# of differing words)

Here, words like this, is, an, the etc. do not really matter
so we could ignore them

stopwords are the useless words



in some cases, removing stopwords might not be helpful
e.g. removing "not" can be dangerous

we need to turn everything to lower case
"PasTa" and "pasta" mean the same

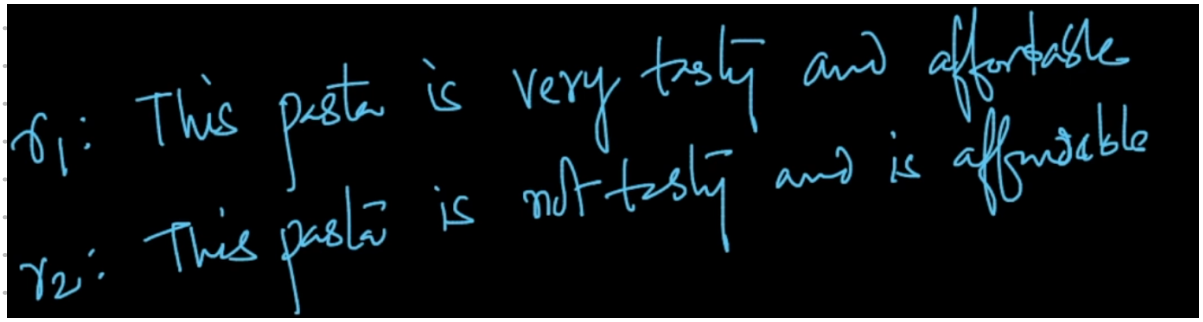
"tasty", "tasteful" and "taste" represent the same thing as well
we could convert all of them to a common form like "taste"

"beautiful", "beauty" -> "beauty"
this is called stemming

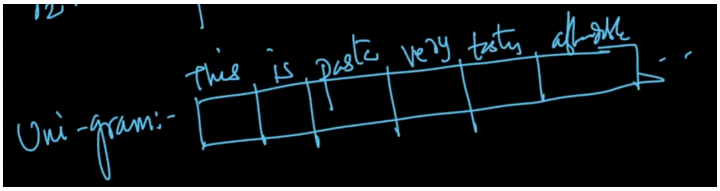
lemmatization:
breaking up a sentence into words
be careful cause not all languages work the same
it is language dependent
also
it is context dependent (New York shouldn't be separated)

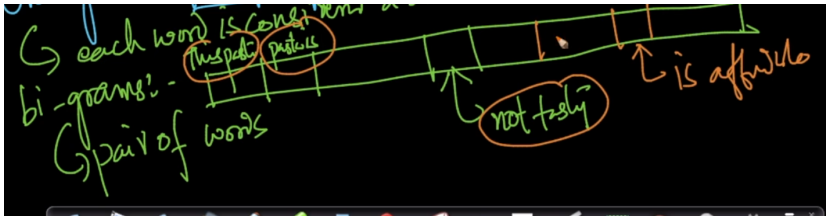
tasty and delicious should be together
BoW doesn't do that by itself

Uni-gram, Bi-gram, and N-gram



In uni-gram:
we use 1 dim for 1 word as we saw above





in bi gram, we use pairs
so not tasty will be 1 dim

n-gram: n words form a dim together

unigram based bag of words completely discards sequence information
"not tasty but cheap" and "tasty but not cheap"
would be the same

but bi gram retains some partial sequence information

but ... > # of tri-grams > # of bi-grams > # of uni-grams

so dimensionality d increases rapidly

An ***n*-gram** is a sequence of *n* adjacent symbols in particular order. The symbols may be *n* adjacent letters (including punctuation marks and blanks), syllables, or rarely whole words found in a language dataset; or adjacent phonemes extracted from a speech-recording dataset, or adjacent base pairs extracted from a genome. They are collected from a text corpus or speech corpus. If Latin numerical prefixes are used, then *n*-gram of size 1 is called a "unigram", size 2 a "bigram" (or, less commonly, a "digram") etc. If, instead of the Latin ones, the English cardinal numbers are furtherly used, then they are called "four-gram", "five-gram", etc. Similarly, using Greek numerical prefixes such as "monomer", "dimer", "trimer", "tetramer", "pentamer", etc., or English cardinal numbers, "one-mer", "two-mer", "three-mer", etc. are used in computational biology, for polymers or oligomers of a known size, called *k*-mers. When the items are words, *n*-grams may also be called *shingles*.[1]

In the context of NLP, the use of *n*-grams allows bag-of-words models to capture information such as word order, which would not be possible in the traditional bag of words setting.

tf-idf: term frequency-inverse document frequency

In information retrieval, **tf–idf** (also **TF*IDF**, **TFIDF**, **TF–IDF**, or **Tf–idf**), short for **term frequency–inverse document frequency**, is a measure of importance of a word to a document in a collection or corpus, adjusted for the fact that some words appear more frequently in general.[1] It was often used as a weighting factor in searches of information retrieval, text mining, and user modeling. A survey conducted in 2015 showed that 83% of text-based recommender systems in digital libraries used tf–idf.[2]

Term frequency [edit]

Term frequency, $\mathrm{tf}(t,d)$, is the relative frequency of term t within document d,

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}},$$

where $f_{t,d}$ is the *raw count* of a term in a document, i.e., the number of times that term t occurs in document d. Note the denominator is simply the total number of terms in document d (counting each occurrence of the same term separately). There are various other ways to define term frequency:[5]:128

- the raw count itself: $\mathrm{tf}(t,d) = f_{t,d}$
- Boolean "frequencies": $\mathrm{tf}(t,d) = 1$ if t occurs in d and 0 otherwise;
- logarithmically scaled frequency: $\mathrm{tf}(t,d) = \log\,(1 + f_{t,d})$;[6]
- augmented frequency, to prevent a bias towards longer documents, e.g. raw frequency divided by the raw frequency of the most frequently occurring term in the document:

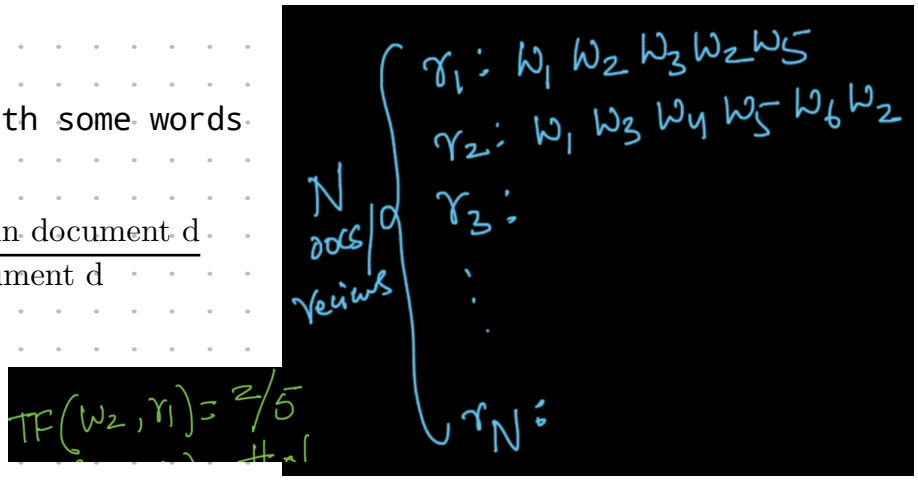
$$\mathrm{tf}(t,d) = 0.5 + 0.5 \cdot \frac{f_{t,d}}{\max\{f_{t',d} : t' \in d\}}$$

Variants of term frequency (tf) weight

weighting scheme	tf weight
binary	$0, 1$
raw count	$f_{t,d}$
term frequency	$f_{t,d} \Big/ \sum_{t' \in d} f_{t',d}$
log normalization	$\log(1 + f_{t,d})$
double normalization 0.5	$0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$
double normalization K	$K + (1 - K)\frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$

say we have n documents, each with some words

$$tf(t,d) = \frac{\#\text{ of times that term t occurs in document d}}{\text{total }\#\text{ of terms in document d}}$$



tf lies betn 0 & 1
we could interpret it as probability

tf(t, d) is how often t occurs in d

Inverse document frequency [edit]

The **inverse document frequency** is a measure of how much information the word provides, i.e., if it is common or rare across all documents. It is the logarithmically scaled inverse fraction of the documents that contain the word (obtained by dividing the total number of documents by the number of documents containing the term, and then taking the logarithm of that quotient):

Variants of inverse document frequency (idf) weight

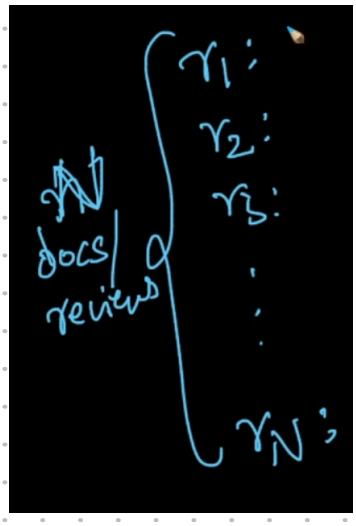
weighting scheme	idf weight ($n_t = \|\{d \in D : t \in d\}\|$)
unary	1
inverse document frequency	$\log \frac{N}{n_t} = -\log \frac{n_t}{N}$
inverse document frequency smooth	$\log\left(\frac{N}{1+n_t}\right)+1$
inverse document frequency max	$\log\left(\frac{\max_{\{t' \in d\}} n_{t'}}{1+n_t}\right)$
probabilistic inverse document frequency	$\log \frac{N-n_t}{n_t}$

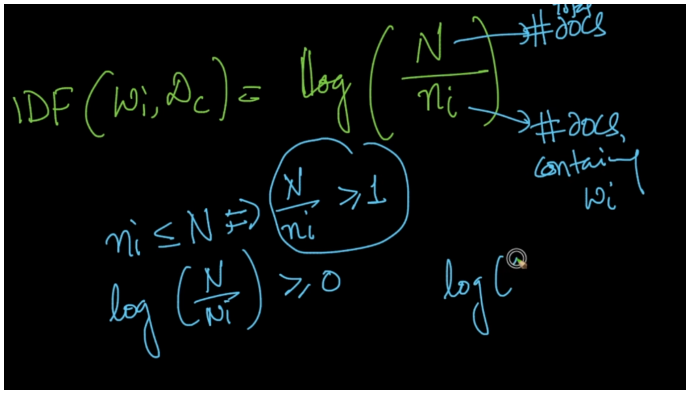
$$\operatorname{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

with

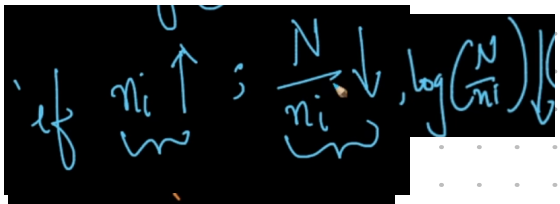
- N: total number of documents in the corpus $N = |D|$
- $|\{d \in D : t \in d\}|$: number of documents where the term t appears (i.e., $\operatorname{tf}(t,d) \neq 0$). If the term is not in the corpus, this will lead to a division-by-zero. It is therefore common to adjust the numerator $1+N$ and denominator to $1+|\{d \in D : t \in d\}|$.

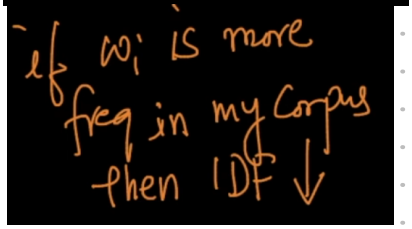
$$idf(t, D) = \log \frac{\#\text{ of documents in corpus D}}{\#\text{ of documents where the term t appears}}$$



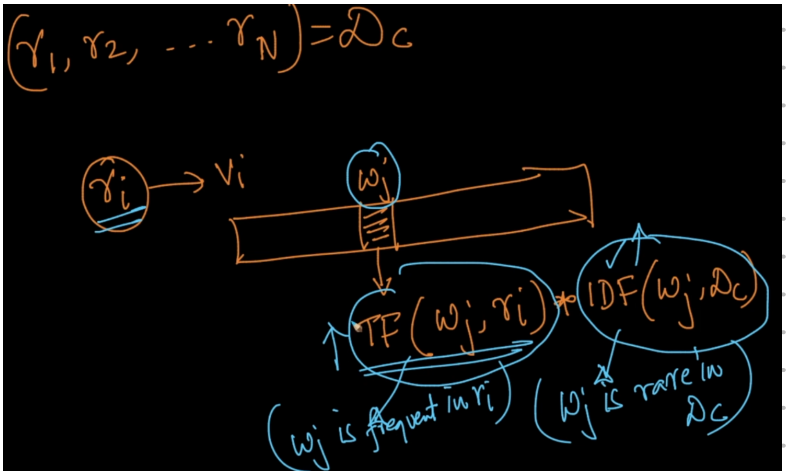


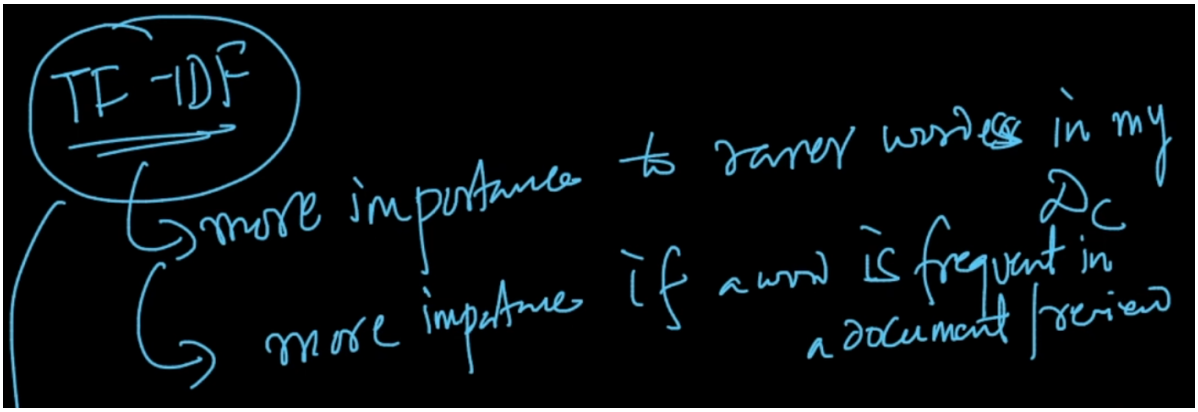
so idf >= 0



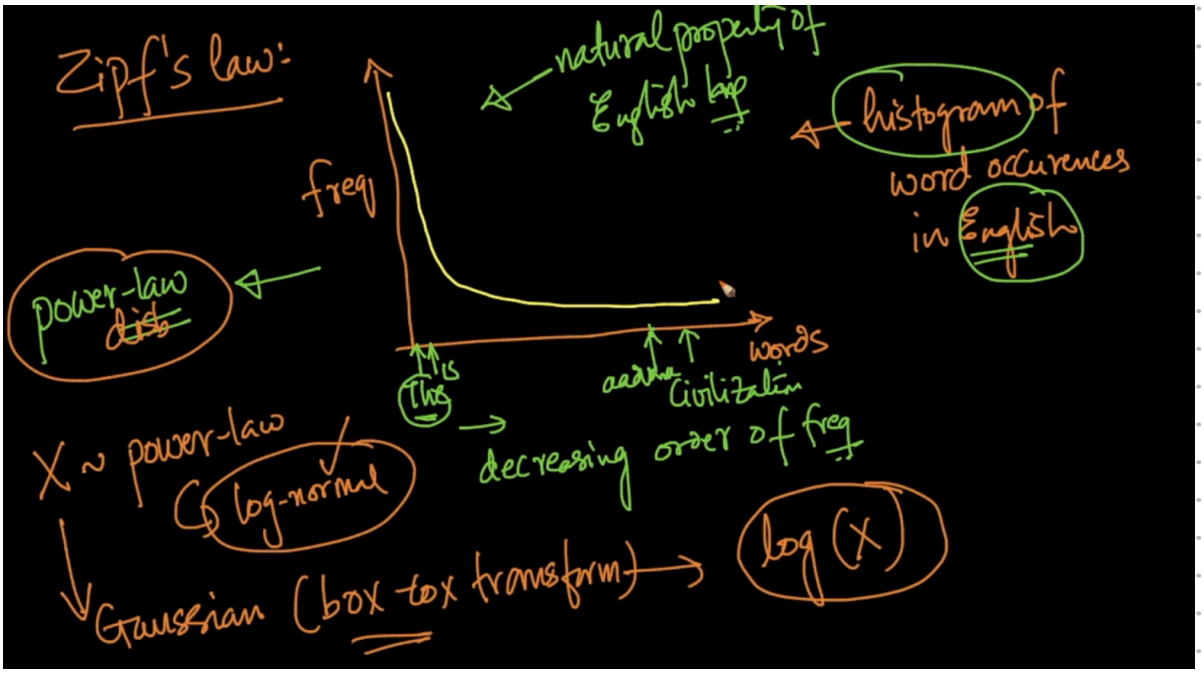


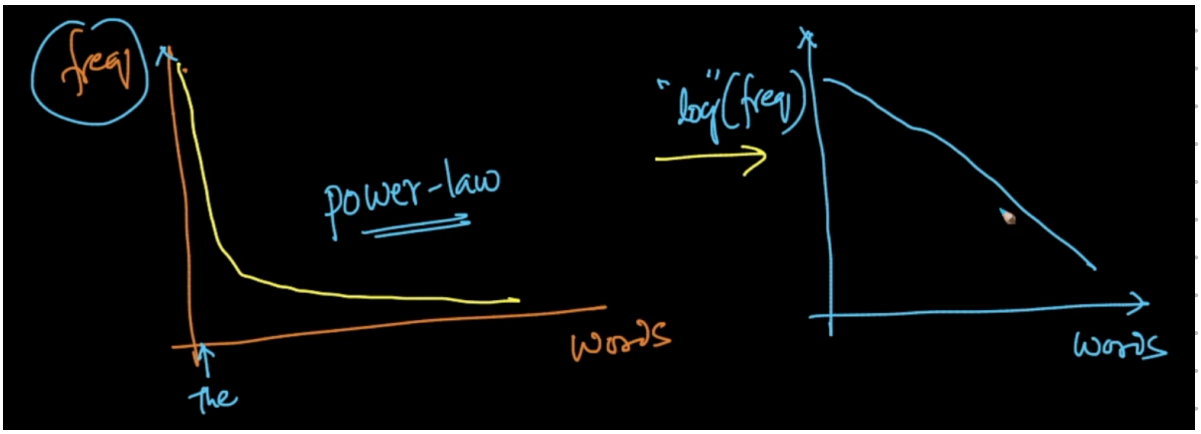
$$\text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D)$$



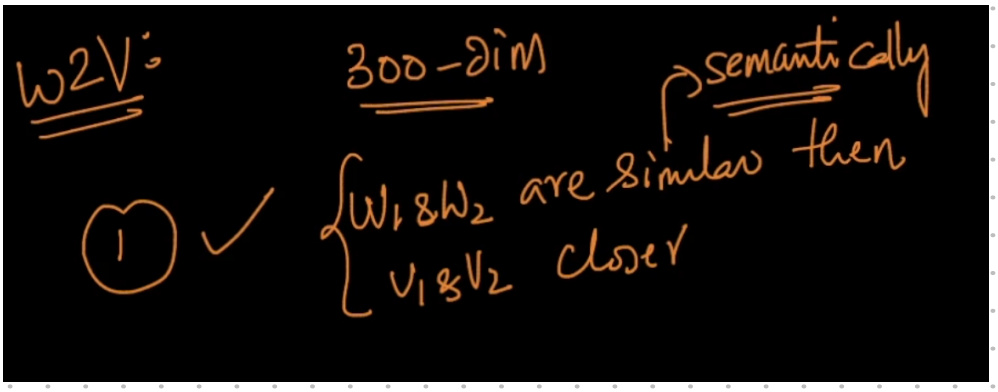


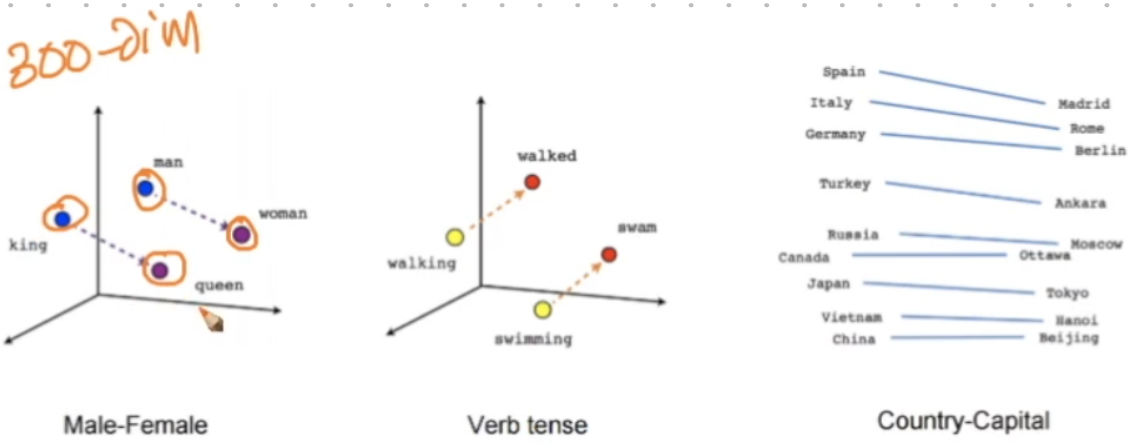
still, tf-idf doesn't consider the semantic meanings of words


Zipf's law:
natural property of English lang
histogram of word occurences in English
freq
power-law
words
This
Civilization
decreasing order of freq
X ~ power-law
log-normal
Gaussian (box-cox transform)
log(X)


freq
power-law
words
The
"log"(freq)
words

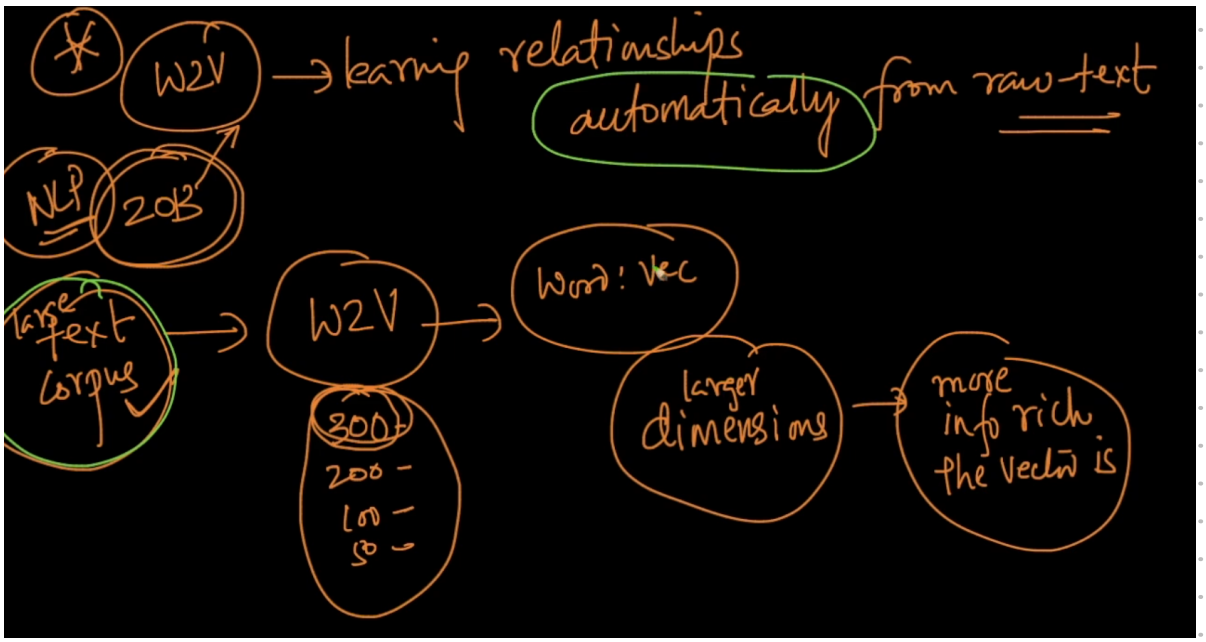
Word2Vec





The basic idea behind Word2Vec is to represent each word as a fixed-size vector,
where the position of the vector in the space reflects the word's semantic meaning.
It's trained on large amounts of text data,
and the model learns to predict the likelihood of a word appearing given its context (Skip-gram model)
or predicting the context given a word (Continuous Bag of Words, CBOW model).

Word2Vec embeddings have interesting properties.
Words with similar meanings end up having similar vector representations,
and vector arithmetic can be used to capture relationships between words.
For example, the vector for "king" minus "man" plus "woman" might be close to the vector for "queen."



Avg Word2Vec

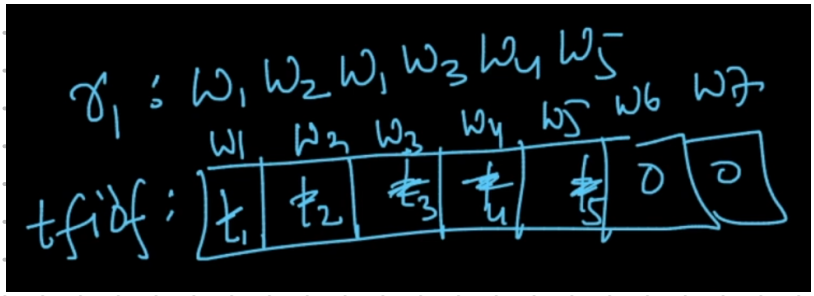
to convert sentence to vector

say r1: w1 w2 w1 w3 w4 w5

we take w2v of individual words and add them,
then we take avg

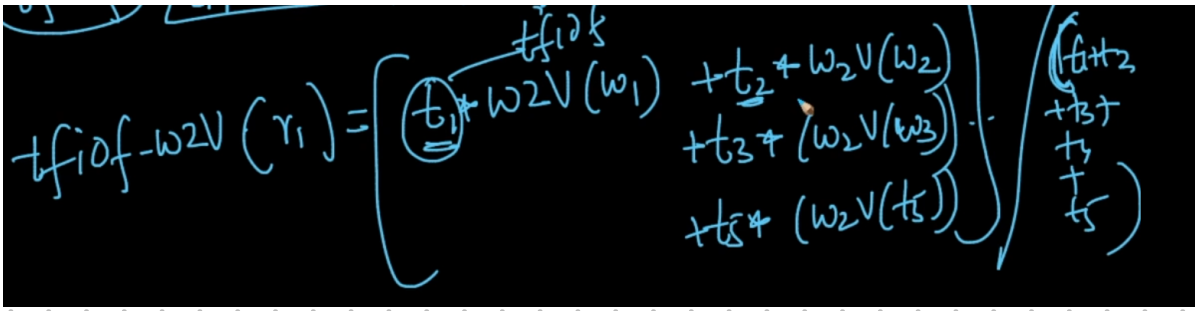
(1/n1)*(w2v(r1) + w2v(r2) + w2v(r1) + w2v(r3) + w2v(r4) + w2v(r5))

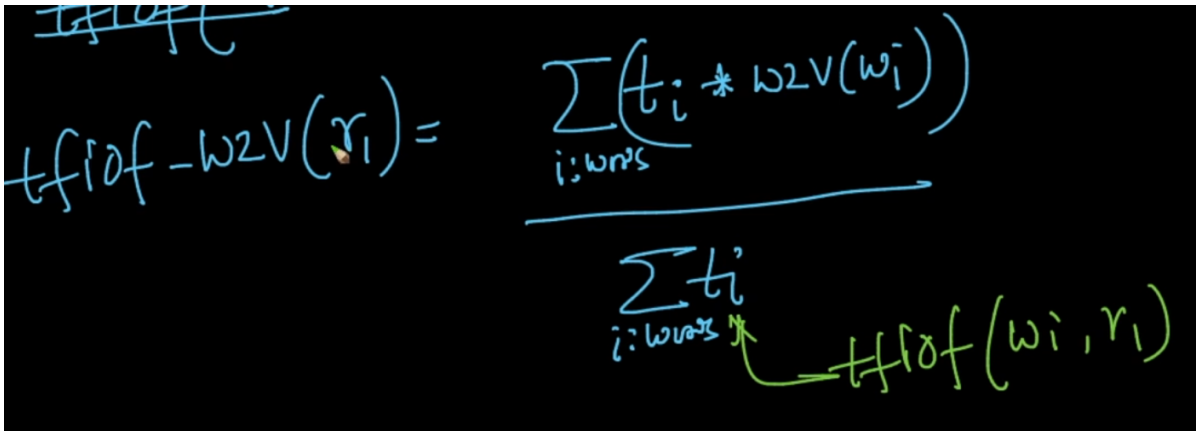
tf-idf W2V



we take tf-idf of words

then tfidf(r1) = [t1*w2v(w1) + t2*w2v(w2) + ...] / sum of tf-idf's





CountVectorizer

Data = ['The', 'quick', 'brown', 'fox', 'jumps', 'over', ' the', 'lazy', 'dog']



	The	quick	brown	fox	jumps	over	lazy	dog
Data	2	1	1	1	1	1	1	1

```python
from sklearn.feature_extraction.text import CountVectorizer

# list of text documents
text = ["John is a good boy. John watches basketball"]

vectorizer = CountVectorizer()
# tokenize and build vocab
vectorizer.fit(text)

print(vectorizer.vocabulary_)

# encode document
vector = vectorizer.transform(text)
# summarize encoded vector
print(vector.shape)
print(vector.toarray())
```

Run

Output 1.59s

```
{'john': 4, 'basketball': 0, 'good': 2, 'is': 3, 'watches': 5, 'boy': 1}
(1, 6)
[[1 1 1 1 2 1]]
```

Bag of Words (BoW)

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
# bag of words
countVectorizer = CountVectorizer()
finalCounts = countVectorizer.fit_transform(final_data["Text"].values)
```

✓ 12.2s

```
finalCounts
```

✓ 0.0s

```
<364131x115281 sparse matrix of type '<class 'numpy.int64'>'
        with 19339928 stored elements in Compressed Sparse Row format>
```

```
finalCounts.shape
```

✓ 0.0s

```
(364131, 115281)
```

sklearn.feature_extraction.text.CountVectorizer

class sklearn.feature_extraction.text.**CountVectorizer**(*, *input='content'*, *encoding='utf-8'*, *decode_error='strict'*, *strip_accents=None*, *lowercase=True*, *preprocessor=None*, *tokenizer=None*, *stop_words=None*, *token_pattern='(?u)\b\w\w+\b'*, *ngram_range=(1, 1)*, *analyzer='word'*, *max_df=1.0*, *min_df=1*, *max_features=None*, *vocabulary=None*, *binary=False*, *dtype=<class 'numpy.int64'>*) [source]

Convert a collection of text documents to a matrix of token counts.

This implementation produces a sparse representation of the counts using scipy.sparse.csr_matrix.

Parameters:

input : *{'filename', 'file', 'content'}, default='content'*

- If `'filename'`, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze.
- If `'file'`, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory.
- If `'content'`, the input is expected to be a sequence of items that can be of type string or byte.

encoding : *str, default='utf-8'*

If bytes or files are given to analyze, this encoding is used to decode.

decode_error : *{'strict', 'ignore', 'replace'}, default='strict'*

Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is 'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.

strip_accents : *{'ascii', 'unicode'} or callable, default=None*

Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed.

Both 'ascii' and 'unicode' use NFKD normalization from `unicodedata.normalize`.

lowercase : *bool, default=True*

Convert all characters to lowercase before tokenizing.

preprocessor : *callable, default=None*

Override the preprocessing (strip_accents and lowercase) stage while preserving the tokenizing and n-grams generation steps. Only applies if `analyzer` is not callable.

tokenizer : *callable, default=None*

Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if `analyzer == 'word'`.

stop_words : *{'english'}, list, default=None*

If 'english', a built-in stop word list for English is used. There are several known issues with 'english' and you should consider an alternative (see Using stop words).

If a list, that list is assumed to contain stop words, all of which will be removed from the resulting tokens. Only applies if `analyzer == 'word'`.

vocabulary : *Mapping or iterable, default=None*

Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or an iterable over terms. If not given, a vocabulary is determined from the input documents. Indices in the mapping should not be repeated and should not have any gap between 0 and the largest index.

binary : *bool, default=False*

If True, all non zero counts are set to 1. This is useful for discrete probabilistic models that model binary events rather than integer counts.

dtype : *dtype, default=np.int64*

Type of the matrix returned by fit_transform() or transform().

ngram_range : *tuple (min_n, max_n), default=(1, 1)*

The lower and upper boundary of the range of n-values for different word n-grams or char n-grams to be extracted. All values of n such such that min_n <= n <= max_n will be used. For example an `ngram_range` of `(1, 1)` means only unigrams, `(1, 2)` means unigrams and bigrams, and `(2, 2)` means only bigrams. Only applies if `analyzer` is not callable.

analyzer : *{'word', 'char', 'char_wb'} or callable, default='word'*

Whether the feature should be made of word n-gram or character n-grams. Option 'char_wb' creates character n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space.

If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input.

Changed in version 0.21.

Since v0.21, if `input` is `filename` or `file`, the data is first read from the file and then passed to the given callable analyzer.

max_df : *float in range [0.0, 1.0] or int, default=1.0*

When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.

min_df : *float in range [0.0, 1.0] or int, default=1*

When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float, the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.

max_features : *int, default=None*

If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used.

This parameter is ignored if vocabulary is not None.

Text Preprocessing: Stemming, Stop-Word Removal, and Lemmatization

1. Begin by removing the html tags
2. Remove any punctuations or limited set of special characters like , or . or # etc.
3. Check if the word is made up of english letters and is not alpha-numeric
4. Check to see if the length of the word is greater than 2 (as it was researched that there is no adjective in 2-letters)
5. Convert the word to lowercase
6. Remove Stopwords
7. Finally Snowball Stemming the word (it was obsereved to be better than Porter Stemming)

identifying HTML tags:

```
i=0;
for sent in final['Text'].values:
    if (len(re.findall('<.*?>', sent))):
        print(i)
        print(sent)
        break;
    i += 1;
```

```
6
I set aside at least an hour each day to read to my son (3 y/o). At this
 point, I consider myself a connoisseur of children's books and this is o
ne of the best. Santa Clause put this under the tree. Since then, we've r
ead it perpetually and he loves it.<br /><br />First, this book taught hi
m the months of the year.<br /><br />Second, it's a pleasure to read. Wel
l suited to 1.5 y/o old to 4+.<br /><br />Very few children's books are w
orth owning. Most should be borrowed from the library. This book, howeve
r, deserves a permanent spot on your shelf. Sendak's best.
```

```
import re
import string

from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem.wordnet import WordNetLemmatizer
```

✓ 0.0s

```
nltk.download("stopwords")
```

✓ 0.8s

```
[nltk_data] Downloading package stopwords to /home/sanved/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.

True
```

```
stop = set(stopwords.words('english'))
sno = nltk.SnowballStemmer('english')
```

✓ 0.0s

```
print(stop)
```

✓ 0.0s

```
{'s', "you're", 'why', 'how', 'are', 'had', 'very', "shan't", 'each',
```

```
sno.stem('beauty'), sno.stem('beautiful')
```

✓ 0.0s

```
('beauti', 'beauti')
```

```
def cleanHTML(document):
    htmlPattern = re.compile('<.*?>')
    cleanText = re.sub(pattern=htmlPattern, repl=' ', string=document)
    return cleanText

def cleanPunctuation(document):
    cleaned = re.sub(pattern=r'[?|!|\'|"|#|.|,|)|(|\\|/]', repl='', string=document)
    return cleaned
```

✓ 0.0s

```
i = 0
str1 = ' '
final_string = []

all_positive_words = []
all_negative_words = []
s = ''
```
✓ 0.0s

```
for sente in final_data["Text"].values:
    filtered_sente = []
    sente = cleanHTML(sente)
    for w in sente.split():
        for cleaned_words in cleanPunctuation(w).split():
            if((cleaned_words.isalpha()) & (len(cleaned_words) > 2)):
                if ( (cleaned_words.lower() not in stop) ):
                    s = (sno.stem(cleaned_words.lower())).encode('utf8')
                    filtered_sente.append(s)
                    if(final_data["Score"].values)[i] == 'positive':
                        all_positive_words.append(s)
                    if(final_data["Score"].values)[i] == 'negative':
                        all_negative_words.append(s)
                else:
                    continue
            else:
                continue

    str1 = b" ".join(filtered_sente)
    final_string.append(str1)
    i += 1
```
✓ 3m 7.5s

```
final_data["cleaned_text"] = final_string
```
✓ 0.0s

```
final_string[1000:1010]
```
✓ 0.0s

```
[b'realli look forward pod base review starbuck good prefer bolder tast imagin surpris order box
 b'peopl enjoy nice espresso latt dinner pod answer easi make mess great flavor without caffein
 b'ive purchas espression espresso classic arabica vote definit arabica flavor bite flavor much
 b'year use starbuck decaf regular singl serv espresso pod final found substitut that econom muc
 b'weve order two box get third home use illi loos ground exclus measur tamp whatnot feel second
 b'order new espresso machin pleas realli shock delici tasti coffe bought base amazon review rig
 b'akitashepard mix come back rub ongo hip problem hurt hima long get day seem greater mobility
 b'look joint supplement activ dog year one tri upset stomach one found seem digest easili tast
 b'unbeliev treat dog anyth get one broken easili smaller piec excel train first ingredi wonder
 b'approv vet multipl qualiti ingredi msmglucosaminecondrioitin canin joint support pet love tas
```

```
final_data.to_csv("final_reviews_data.csv")
```
✓ 19.5s

```
conn = sqlite3.connect('final_data.sqlite')
c = conn.cursor()
conn.text_factory = str
final_data.to_sql("Reviews", conn, schema=None, if_exists="replace")
```
✓ 10.9s

```
364131
```

```
final_data.iloc[1000:1010][["Text", "cleaned_text"]]
```
✓ 0.0s

	Text	cleaned_text
333933	I was really looking forward to these pods bas...	b'realli look forward pod base review starbuck...
333928	For people that enjoy a nice espresso or latte...	b'peopl enjoy nice espresso latt dinner pod an...
333924	I've purchased both the Espressione Espresso (...	b'ive purchas espression espresso classic arab...
333923	After years of using Starbuck's decaf and regu...	b'year use starbuck decaf regular singl serv e...
333929	We've ordered two of these boxes and I'm about...	b'weve order two box get third home use illi l...
333925	We ordered these to go with our new espresso m...	b'order new espresso machin pleas realli shock...
392422	My Akita/Shepard mix comes to me for "back rub...	b'akitashepard mix come back rub ongo hip prob...
392423	I have looked for a joint supplement for my ve...	b'look joint supplement activ dog year one tri...
392420	This is the most unbelievable treat. My dog w...	b'unbeliev treat dog anyth get one broken easi...
392421	approved by my vet as having multiple quality ...	b'approv vet multipl qualiti ingredi msmglucos...

Bi-grams and n-grams

```
freq_dist_positive = nltk.FreqDist(all_positive_words)
freq_dist_negative = nltk.FreqDist(all_negative_words)

print(freq_dist_positive.most_common(5))
print(freq_dist_negative.most_common(5))
```

✓ 20.0s

```
[(b'like', 139060), (b'tast', 128063), (b'good', 112005), (b'flavor', 108640), (b'love', 107010)]
[(b'tast', 34299), (b'like', 32219), (b'product', 28001), (b'one', 20518), (b'flavor', 19367)]
```

here words overlap in positive and negative.
the word "like" is in both positive and negative reviews.
it probably occurs in phrases like "did not like" or sth. like that.
that's why we need bi-gram and n-gram.

sklearn.feature_extraction.text.CountVectorizer

class sklearn.feature_extraction.text.**CountVectorizer**(*, *input='content'*, *encoding='utf-8'*, *decode_error='strict'*, *strip_accents=None*, *lowercase=True*, *preprocessor=None*, *tokenizer=None*, *stop_words=None*, *token_pattern='(?u)\b\w\w+\b'*, *ngram_range=(1, 1)*, *analyzer='word'*, *max_df=1.0*, *min_df=1*, *max_features=None*, *vocabulary=None*, *binary=False*, *dtype=<class 'numpy.int64'>*) [source]

ngram_range : *tuple (min_n, max_n), default=(1, 1)*

The lower and upper boundary of the range of n-values for different word n-grams or char n-grams to be extracted. All values of n such such that min_n <= n <= max_n will be used. For example an `ngram_range` of `(1, 1)` means only unigrams, `(1, 2)` means unigrams and bigrams, and `(2, 2)` means only bigrams. Only applies if `analyzer` is not callable.

btw we removed the word "not" as a stopword
we should not have. it carries a huge significance.

```
count_vect = CountVectorizer(ngram_range=(1, 2))
final_bigram_counts = count_vect.fit_transform(final_data["Text"].values)
```

✓ 2m 27.2s

```
final_bigram_counts.shape
```

✓ 0.0s

```
(364131, 2910192)
```

too much more dimensions than unigram

tf-idf

```
tfidf_vect = TfidfVectorizer(ngram_range=(1, 2))
final_tfidf = tfidf_vect.fit_transform(final_data["Text"].values)
```

✓ 2m 26.0s

```
final_tfidf.shape
```

✓ 0.0s

```
(364131, 2910192)
```

```
(tfidf_vect.get_feature_names_out())[10000:10010]
```

✓ 3.4s

```
array(['14 count', '14 country', '14 crackers', '14 credit', '14 crude',
       '14 cup', '14 cupcakes', '14 cups', '14 currants', '14 dad'],
      dtype=object)
```

```
(tfidf_vect.get_feature_names_out())[100000:100010]
```

✓ 3.4s

```
array(['ales until', 'ales ve', 'ales would', 'ales you', 'alessandra',
       'alessandra ambrosia', 'alessi', 'alessi added', 'alessi also',
       'alessi and'], dtype=object)
```

```
# source: https://buhrmann.github.io/tfidf-analysis.html
def top_tfidf_feats(row, features, top_n=25):
    ''' Get top n tfidf values in row and return them with their correspond
    topn_ids = np.argsort(row)[::-1][:top_n]
    top_feats = [(features[i], row[i]) for i in topn_ids]
    df = pd.DataFrame(top_feats)
    df.columns = ['feature', 'tfidf']
    return df

top_tfidf = top_tfidf_feats(final_tf_idf[1,:].toarray()[0],features,25)
```

```
top_tfidf
```

	feature	tfidf
0	sendak books	0.173437
1	rosie movie	0.173437
2	paperbacks seem	0.173437
3	cover version	0.173437
4	these sendak	0.173437
5	the paperbacks	0.173437
6	pages open	0.173437
7	really rosie	0.168074
8	incorporates them	0.168074
9	paperbacks	0.168074
10	however miss	0.164269

Word2Vec

```
# Using Google News Word2Vectors
from gensim.models import Word2Vec
from gensim.models import KeyedVectors
import pickle

# in this project we are using a pretrained model by google
# its 3.3G file, once you load this into your memory
# it occupies ~9Gb, so please do this step only if you have >12G of ram
# we will provide a pickle file wich contains a dict ,
# and it contains all our courpus words as keys and  model[word] as values
# To use this code-snippet, download "GoogleNews-vectors-negative300.bin"
# from https://drive.google.com/file/d/0B7XkCwpI5KDYNlNUTTlSS21pQmM/edit
# it's 1.9GB in size.


model = KeyedVectors.load_word2vec_format('GoogleNews-vectors-negative300.b
```

```
model.wv['computer']
```

```
array([  1.07421875e-01,  -2.01171875e-01,   1.23046875e-01,
         2.11914062e-01,  -9.13085938e-02,   2.16796875e-01,
        -1.31835938e-01,   8.30078125e-02,   2.02148438e-01,
         4.78515625e-02,   3.66210938e-02,  -2.45361328e-02,
         2.39257812e-02,  -1.60156250e-01,  -2.61230469e-02,
         9.71679688e-02,  -6.34765625e-02,   1.84570312e-01,
         1.70898438e-01,  -1.63085938e-01,  -1.09375000e-01,
         1.49414062e-01,  -4.65393066e-04,   9.61914062e-02,
         1.68945312e-01,   2.60925293e-03,   8.93554688e-02,
         6.49414062e-02,   3.56445312e-02,  -6.93359375e-02,
```

```
model.wv.similarity('woman', 'man')
```

```
0.76640122309953518
```

```
model.wv.most_similar('woman')
```

```
[('man', 0.7664012312889099),
 ('girl', 0.7494641542434692),
 ('teenage_girl', 0.7336830496788025),
 ('teenager', 0.6317086219787598),
 ('lady', 0.6288787126541138),
 ('teenaged_girl', 0.6141784191131592),
 ('mother', 0.607630729675293),
 ('policewoman', 0.6069462299346924),
 ('boy', 0.5975908041000366),
 ('Woman', 0.5770982503890991)]
```

```
model.wv.most_similar('tasty')  # "tasti"
```

```
[('delicious', 0.8730389475822449),
 ('scrumptious', 0.8007042407989502),
 ('yummy', 0.7856923341751099),
 ('flavorful', 0.7420164346694946),
 ('delectable', 0.7385422587394714),
 ('juicy_flavorful', 0.7114803791046143),
 ('appetizing', 0.701721727848053),
 ('crunchy_salty', 0.7012301087379456),
 ('flavourful', 0.6912214159965515),
 ('flavoursome', 0.6857703328132629)]
```

```
# Train your own Word2Vec model using your own text corpus
import gensim
i=0
list_of_sent=[]
for sent in final['Text'].values:
    filtered_sentence=[]
    sent=cleanhtml(sent)
    for w in sent.split():
        for cleaned_words in cleanpunc(w).split():
            if(cleaned_words.isalpha()):
                filtered_sentence.append(cleaned_words.lower())
            else:
                continue
    list_of_sent.append(filtered_sentence)
```

```
w2v_model=gensim.models.Word2Vec(list_of_sent,min_count=5,size=50, workers=4)
```

```
words = list(w2v_model.wv.vocab)
print(len(words))
```

```
33783
```

```
w2v_model.wv.most_similar('tasty')
```

```
[('tastey', 0.909038245677948),
 ('satisfying', 0.8556904792785645)
 ('yummy', 0.8543208837509155),
 ('filling', 0.8233586549758911),
 ('delicious', 0.8229926228523254)
```

```
w2v_model.wv.most_similar('tasty')
```

```
[('tastey', 0.909038245677948),
 ('satisfying', 0.8556904792785645),
 ('yummy', 0.8543208837509155),
 ('filling', 0.8233586549758911),
 ('delicious', 0.8229926228523254),
 ('flavorful', 0.8061250448226929),
 ('addicting', 0.771919846534729),
 ('delish', 0.7653154730796814),
 ('nutritious', 0.7626035213470459),
 ('tasteful', 0.7547359466552734)]
```

```
w2v_model.wv.most_similar('like')
```

```
[('resemble', 0.7139369249343872),
 ('mean', 0.651788055896759),
 ('prefer', 0.646423876285553),
 ('dislike', 0.6413203477859497),
 ('overpower', 0.6197512745857239),
 ('think', 0.5993291735649109),
 ('overwhelm', 0.5929116606712341),
 ('enjoy', 0.5895069241523743),
 ('gross', 0.5853881239891052),
 ('alright', 0.5824472308158875)]
```

```
count_vect_feat = count_vect.get_feature_names() #
count_vect_feat.index('like')
print(count_vect_feat[64055])
```

```
like
```

average w2v and tf-idf weighted w2v

```
# average Word2Vec
# compute average word2vec for each review.
sent_vectors = []; # the avg-w2v for each sentence/review is stored in this
for sent in list_of_sent: # for each review/sentence
    sent_vec = np.zeros(50) # as word vectors are of zero length
    cnt_words =0; # num of words with a valid vector in the sentence/review
    for word in sent: # for each word in a review/sentence
        try:
            vec = w2v_model.wv[word]
            sent_vec += vec
            cnt_words += 1
        except:
            pass
    sent_vec /= cnt_words
    sent_vectors.append(sent_vec)
print(len(sent_vectors))
print(len(sent_vectors[0]))
```

```
/Library/Frameworks/Python.framework/Versions/3.6/lib/python3.6/site-pack
ages/ipykernel_launcher.py:14: RuntimeWarning: invalid value encountered
 in true_divide
```

```
364171
50
```

```
# TF-IDF weighted Word2Vec
tfidf_feat = tf_idf_vect.get_feature_names() # tfidf words/col-names
# final_tf_idf is the sparse matrix with row= sentence, col=word and cell_v

tfidf_sent_vectors = []; # the tfidf-w2v for each sentence/review is stored
row=0;
for sent in list_of_sent: # for each review/sentence
    sent_vec = np.zeros(50) # as word vectors are of zero length
    weight_sum =0; # num of words with a valid vector in the sentence/revie
    for word in sent: # for each word in a review/sentence
        try:
            vec = w2v_model.wv[word]
            # obtain the tf_idfidf of a word in a sentence/review
            tfidf = final_tf_idf[row, tfidf_feat.index(word)]
            sent_vec += (vec * tf_idf)
            weight_sum += tf_idf
        except:
            pass
    sent_vec /= weight_sum
    tfidf_sent_vectors.append(sent_vec)
    row += 1
```